

PROYECTO INTERMODULAR



Técnico Superior en Desarrollo de Aplicaciones
Multiplataforma

Clef

Estudio de cifrado y seguridad

Autor: Óscar Padrino López

Autor: Adrián Muñoz Saiz

Autor: José Manuel Perdices Hernández

Director: César Rodríguez Casero

Tutor curso: Diana Pastor Lazcano

Tutor proyecto: Alejandro De Acuña Jiménez

Alcalá de Henares, mayo de 2026

Clef



Anexo A

Contenido

1. Introducción: por qué este documento existe.....	3
2. Fundamentos: criptografía, cifrado y hash.....	3
2.1 Criptografía.....	4
2.2 Cifrado	4
2.3 Hash	5
3. Tipos de cifrado: simétrico, asimétrico e híbrido.....	5
3.1 Cifrado simétrico.....	6
3.2 Cifrado asimétrico.....	7
3.3 Cifrado híbrido	8
3.4 Tabla comparativa.....	9
3.5 ¿qué usa Clef?	9
4. Funciones hash y derivación de claves (KDF)	10
4.1 ¿Qué es un hash?	10
4.2 El problema de hashear contraseñas directamente	11
4.3 La solución: KDFs (Key Derivation Functions)	12
4.4 ¿Qué KDF usa Clef?	13
4.5 ¿Qué obtiene PBKDF2?	14
5. La salt: qué es y por qué existe	14
5.1 El ataque que la salt previene	14
5.2 La idea de la salt	15
5.3 ¿La salt tiene que ser secreta?.....	16
5.4 La salt en Clef.....	16
6. Arquitecturas de seguridad en gestores de contraseñas.....	17
6.1 Modelo cliente-servidor tradicional.....	17
6.2 Modelo end-to-end encryption (E2EE)	18
6.3 Modelo Zero-Knowledge (ZK)	18
6.4 Comparativa	19
6.5 Otros modelos relevantes	20
7. Por qué Clef es Zero-Knowledge	20
7.1 La razón filosófica.....	20
7.2 La razón técnica concreta en Clef	21
7.3 Ventajas	21

7.4 Desventajas (y son reales)	22
8. Algoritmos concretos implementados en Clef	23
8.1 AES-256-GCM (cifrado simétrico autenticado)	23
8.2 PBKDF2-HMAC-SHA256 (derivación de clave)	25
8.3 SecureRandom (generación de aleatoriedad)	26
8.4 Key Wrapping: la jerarquía de claves de Clef	26
8.5 Android Keystore (TEE para biometría)	27
9. Flujo criptográfico completo de la aplicación.....	28
9.1 Registro (primer uso, <i>KeyManager.register()</i>)	28
9.2 Login diario (<i>KeyManager.login()</i>)	29
9.3 Guardar una credencial nueva (<i>AddItemDialog</i>)	30
9.4 Recuperación con PUK (<i>KeyManager.recoverWithPuk()</i>)	30
9.5 Auto-Lock.....	31
9.6 Biometría (cuando está activada)	31
10. Modelo de amenazas y mitigaciones	32
10.1 Ataque de fuerza bruta online	32
10.2 Ataque de fuerza bruta offline	33
10.3 Ataque Man-in-the-Middle (MitM)	34
10.4 Robo del dispositivo	34
10.5 Phishing	35
10.6 Manipulación de la base de datos	36
10.7 Replay attacks	36
11. El caso crítico: “¿qué pasa si roban Firestore?”	37
12. Limitaciones honestas del modelo	39
12.1 Dependencia de la calidad de la contraseña maestra.....	40
12.2 Iteraciones de PBKDF2.....	40
12.3 Cliente comprometido = todo comprometido	40
12.4 Metadatos visibles	41
12.5 Recuperación dependiente del PUK	41
12.6 PUK temporalmente en memoria	41
12.7 No protege contra el sistema operativo	42
13. Bibliografía y referencias.....	42
Apéndice A: Glosario rápido	45

1. Introducción: por qué este documento existe

Cuando uno construye un gestor de contraseñas, no basta con que “funcione”. Funcionar lo hacen muchos: hay aplicaciones que guardan tus credenciales en una base de datos en texto plano y se anuncian como “seguras”. El problema aparece el día que esa base de datos se filtra — y entonces *todas* las contraseñas de *todos* los usuarios quedan expuestas de golpe.

Este documento explica:

- Qué es la criptografía y los distintos tipos de cifrado que existen.
 - Cuál usa Clef y por qué.
 - Qué arquitecturas de seguridad existen en gestores de contraseñas y por qué se eligió Zero-Knowledge.
 - Qué ataques teóricos puede recibir la aplicación y cómo se mitigan.
 - Qué pasa exactamente si Firestore se ve comprometido.
 - Las limitaciones honestas del modelo, porque ningún sistema es perfecto y reconocer esos límites es parte de defenderlo bien.
-

2. Fundamentos: criptografía, cifrado y hash

Antes de hablar de Clef hay que tener claros tres conceptos que la gente mezcla constantemente.

2.1 Criptografía

La criptografía es la disciplina que estudia técnicas matemáticas para **proteger información**. Persigue cuatro objetivos clásicos:

Objetivo	Qué garantiza
Confidencialidad	Que solo quien debe leer un mensaje pueda leerlo.
Integridad	Que el mensaje no haya sido modificado en el camino.
Autenticación	Que el mensaje venga realmente de quien dice venir.
No repudio	Que el emisor no pueda negar después que lo envió.

Clef se centra en los tres primeros. El no repudio es más típico de firmas digitales y blockchain.

2.2 Cifrado

Cifrado es **una técnica concreta dentro de la criptografía**: transformar información legible en información ilegible (texto cifrado), de manera que la operación sea reversible *si tienes la clave*.

Texto plano → [FUNCIÓN DE CIFRADO + CLAVE] → Texto cifrado
Texto cifrado → [FUNCIÓN DE DESCIFRADO + CLAVE] → Texto plano

Lo importante: **el cifrado es reversible**. Para eso sirve.

3. Tipos de cifrado: simétrico, asimétrico e híbrido

2.3 Hash

Una función hash hace algo distinto: toma una entrada de cualquier tamaño y produce una salida de tamaño fijo, **de manera que la operación NO es reversible**.

```
"contraseña123" → [ FUNCIÓN HASH ] → 9b8769a4a742959a2d0298c36fb70623f2
```

Si te dan el hash, no puedes recuperar la entrada original. Por eso un hash sirve para verificar contraseñas (guardas el hash, no la contraseña), pero **no sirve para guardar datos que necesites recuperar**. Si guardases tus contraseñas como hash, no podrías mostrarlas nunca al usuario — tendría que adivinarlas él mismo.

Confusión típica: “tu contraseña está hasheada” ≠ “tu contraseña está cifrada”. Un servicio que hashea tu contraseña al registrarte hace lo correcto. Un gestor de contraseñas que solo hashee las contraseñas que guarda sería inútil: nunca podrías recuperarlas.

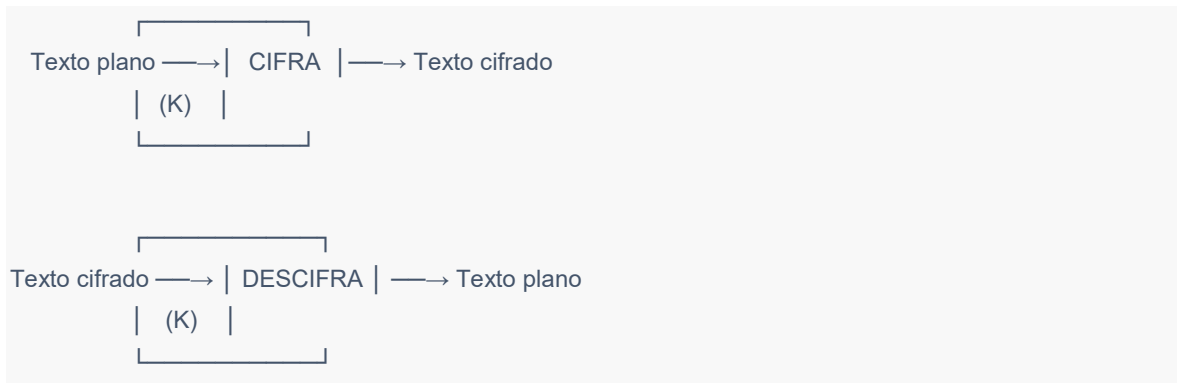
Clef usa **las dos cosas**: hashes (a través de PBKDF2) para procesar la contraseña maestra, y cifrado (AES-256-GCM) para proteger los datos.

3. Tipos de cifrado: simétrico, asimétrico e híbrido

Esta es la pregunta clásica de ciberseguridad. Vamos por partes.

3.1 Cifrado simétrico

Una sola clave sirve tanto para cifrar como para descifrar. Es como una caja fuerte con una llave: la misma llave la cierra y la abre.



Algoritmos típicos: AES (Advanced Encryption Standard), ChaCha20, 3DES (obsoleto), DES (obsoleto y roto desde hace décadas).

Ventajas: - Muy rápido. AES-256 cifra a velocidades de gigabytes por segundo en hardware moderno (la mayoría de procesadores tienen instrucciones AES-NI dedicadas). - Claves cortas y matemáticamente robustas: una clave AES-256 ofrece 2^{256} combinaciones posibles. Para hacerse una idea: el universo tiene unos 10^{80} átomos; 2^{256} son unos 10^{77} valores. La fuerza bruta no es viable ni con todos los ordenadores del planeta funcionando hasta el fin del universo.

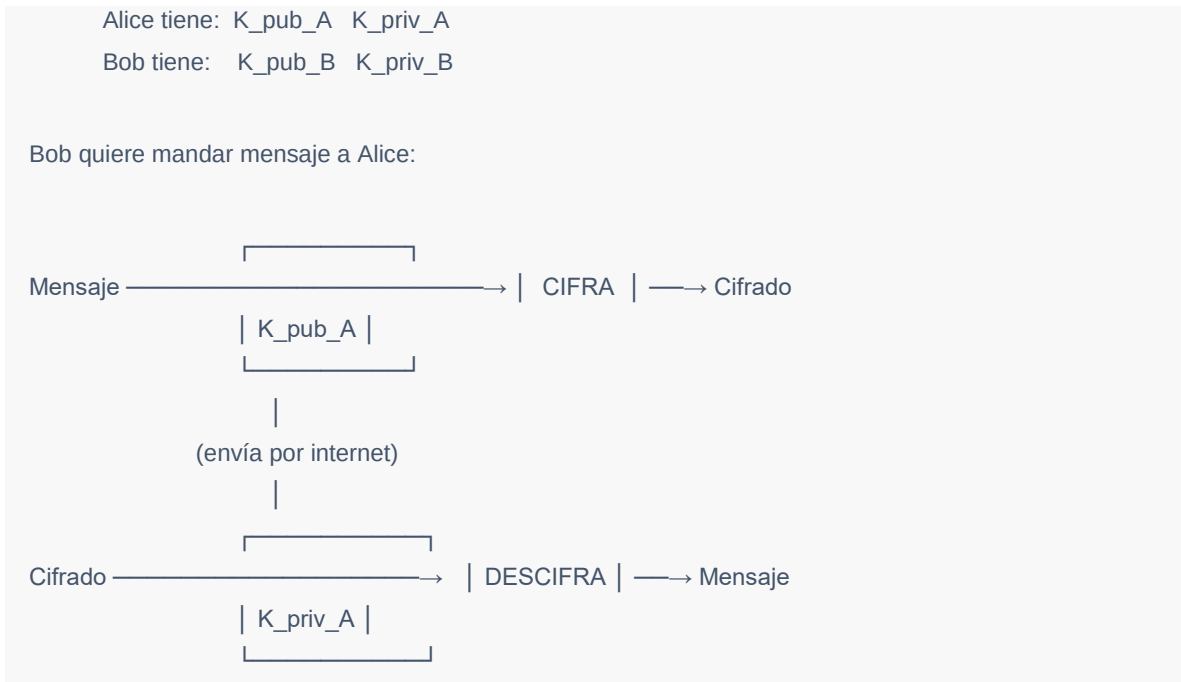
Desventaja crítica: - **El problema del intercambio de clave.** Si tú y yo queremos comunicarnos cifrado con AES, primero tenemos que ponernos de acuerdo en la clave. ¿Pero cómo te paso la clave de manera segura sin que un tercero la intercepte? Si yo te la mando por internet sin cifrar, cualquiera la lee y se acabó la confidencialidad.

Este problema es lo que dio origen al cifrado asimétrico.

3. Tipos de cifrado: simétrico, asimétrico e híbrido

3.2 Cifrado asimétrico

Cada usuario tiene **dos claves matemáticamente relacionadas**: una pública y una privada. Lo que cifra con una solo se descifra con la otra.



La clave pública la puedes publicar en cualquier sitio: en tu web, en un servidor, donde quieras. La privada nunca sale de tu dispositivo. Cuando alguien te quiere mandar algo cifrado, usa tu clave pública. Solo tú, con la privada, puedes leerlo.

Algoritmos típicos: RSA, ECC (Elliptic Curve Cryptography), Curve25519, Ed25519.

Ventajas: - Resuelve el problema del intercambio de claves: ya no hace falta compartir un secreto previo. - Permite firmas digitales: si yo cifro algo con mi clave privada, cualquiera puede verificar con mi clave pública que lo escribí yo.

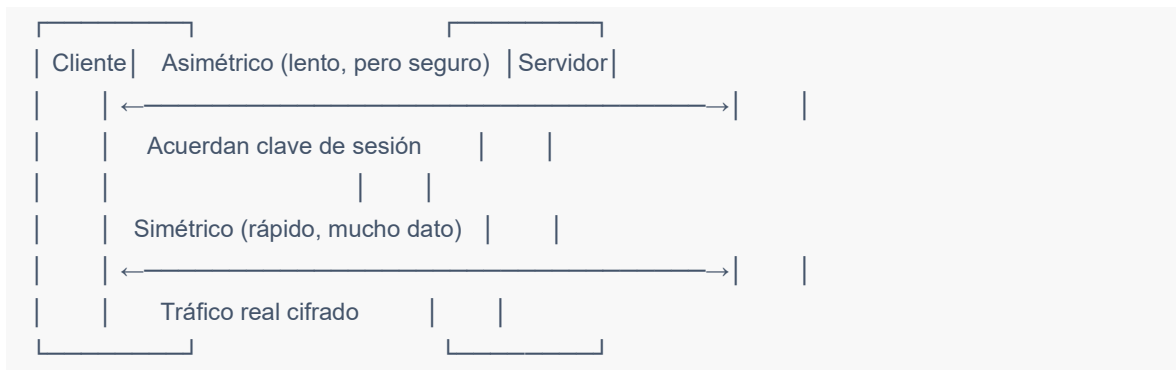
Desventajas: - **Mucho más lento.** RSA es órdenes de magnitud más lento que AES. No es práctico para cifrar grandes volúmenes de datos. - Las claves son mucho más largas. Una clave RSA "segura" hoy son 3072 bits o 4096 bits. ECC es más eficiente (256 bits dan seguridad equivalente a RSA-3072), pero sigue siendo más pesado que el simétrico.

3.3 Cifrado híbrido

Como cada tipo tiene sus virtudes y defectos, en la práctica casi nadie usa solo uno. Lo habitual es **combinar ambos**: usar el asimétrico para resolver el problema de intercambio de clave, y el simétrico para cifrar los datos en sí.

Es exactamente lo que hace **HTTPS / TLS**: cuando entras en una web, tu navegador y el servidor:

1. Intercambian sus claves públicas.
2. Negocian usando criptografía asimétrica una **clave de sesión** simétrica.
3. A partir de ese momento, todo el tráfico real usa cifrado simétrico (porque es rápido), pero la clave de sesión solo la conocen ellos dos.



Otros ejemplos: PGP/GPG para correo, Signal para mensajería, SSH para acceso remoto. Todos son híbridos.

3.4 Tabla comparativa

Característica	Simétrico	Asimétrico	Híbrido
Velocidad	Muy alta	Baja	Alta (datos cifrados con simétrico)
Tamaño de clave	256 bits	3072+ bits (RSA), 256 bits (ECC)	Ambos
Problema clave	Intercambio	Más lento	Resuelve ambos
Uso típico	Cifrar archivos, bases de datos	Intercambio de claves, firmas	Comunicaciones (HTTPS, TLS)
Ejemplos	AES, ChaCha20	RSA, ECC	TLS, PGP, Signal

3.5 ¿qué usa Clef?

Clef usa **cifrado simétrico puro**, concretamente **AES-256-GCM**.

Esto al principio puede sorprender porque suena como si fuera “menos seguro” que un sistema híbrido. **No lo es**. La elección está pensada y tiene sentido por el modelo de uso de la aplicación. La razón es la siguiente:

Clef no es un sistema de comunicación entre dos partes que no se conocen. Es un sistema donde **una sola persona** —el usuario— necesita cifrar sus propios datos para que **solo ella** pueda leerlos después.

El cifrado asimétrico resuelve el problema de “cómo te mando un secreto sin tener que compartir antes una clave contigo”. Pero en Clef no hay un “tú” y un “yo”. Hay solo un usuario que se está mandando datos a sí mismo a través del tiempo (tu yo

de hoy guarda una contraseña que tu yo de mañana querrá leer). Para ese caso de uso, el cifrado simétrico es **lo adecuado**: rápido, robusto y suficiente.

Lo que sí tiene Clef que es especial, y se confunde a veces con cifrado híbrido, es una **jerarquía de claves simétricas** llamada **Key Wrapping** (envoltura de claves). Lo veremos en detalle en la sección 8.

4. Funciones hash y derivación de claves (KDF)

4.1 ¿Qué es un hash?

Una **función hash criptográfica** es una función matemática que cumple cuatro propiedades:

1. **Determinista**: la misma entrada produce siempre el mismo hash.
2. **Tamaño fijo**: la salida tiene siempre la misma longitud, independientemente de lo que entre.
3. **No reversible** dado un hash, es computacionalmente inviable encontrar la entrada que lo produjo.
4. **Resistente a colisiones**: es computacionalmente inviable encontrar dos entradas distintas que produzcan el mismo hash.

Ejemplo con SHA-256:

```
"hola"      → 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
"hola mundo" → 0bfe935e70c321c7ca3afc75ce0d0ca2f98b5422e008bb31c00c6d7f1f1c0ad6
"holaa"     → 87298cc2f31fba73181ea2a9e6ef10dce21ed95e98bdac9c4e1504ea16f486e4
```

Notas dos cosas importantes:

- Cambiar **un solo carácter** ("hola" → "holaa") produce un hash radicalmente distinto. Eso se llama **efecto avalancha**, y es una propiedad deseada.

4. Funciones hash y derivación de claves (KDF)

- La salida siempre tiene 64 caracteres hexadecimales = 32 bytes = 256 bits.

Algoritmos hash comunes:

Algoritmo	Tamaño salida	Estado
MD5	128 bits	Roto. No usar para nada criptográfico.
SHA-1	160 bits	Roto (Google demostró colisiones en 2017).
SHA-256	256 bits	Estándar actual. Robusto.
SHA-512	512 bits	Estándar actual. Robusto.
SHA-3	Variable	Nuevo estándar NIST 2015.
BLAKE2 BLAKE3	/ Variable	Modernos, muy rápidos.

4.2 El problema de hashear contraseñas directamente

Aquí viene un matiz fundamental que muchos sistemas se saltan y luego lo pagan caro.

Si tú coges una contraseña de usuario y la pasas directamente por SHA-256, has hecho algo *casi* bien. ¿Qué falla?

El problema 1: las contraseñas de la gente son predecibles.

Las top 10 contraseñas más usadas del mundo siguen siendo cosas como `123456`, `password`, `qwerty`. Un atacante puede precalcular el SHA-256 de las 10.000 contraseñas más comunes y guardarlas en una tabla. Cuando le caiga tu base de datos, busca tus hashes en la tabla y, si tu usuario usó `password123`, lo identifica al instante. Esto se llama **ataque por diccionario**.

Una variante más sofisticada son las **rainbow tables**: tablas precalculadas optimizadas que pueden cubrir miles de millones de contraseñas con un coste de almacenamiento manejable.

El problema 2: SHA-256 es rapidísimo.

Una GPU moderna calcula varios miles de millones de hashes SHA-256 por segundo. Eso significa que un atacante puede probar **toda** una lista de contraseñas filtradas en cuestión de minutos.

4.3 La solución: KDFs (Key Derivation Functions)

Para protegerse de eso se inventaron los **KDF**: funciones de derivación de claves que toman una contraseña y la transforman en una clave criptográfica de manera **deliberadamente lenta y costosa**.

Los KDFs hacen dos cosas:

1. Aplican el hash interno **muchísimas veces** (decenas o cientos de miles de iteraciones), de manera que aunque cada hash individual sea rápido, el total se hace lento.
2. Mezclan la contraseña con un valor aleatorio único llamado **salt**, que destruye los ataques por diccionario y rainbow tables.

Familias de KDF más conocidas:

KDF	Año	Característica
PBKDF2	2000	El clásico. Lento por iteraciones. Estándar NIST.
bcrypt	1999	Lento + uso intensivo de memoria moderado.
scrypt	2009	Lento + uso intensivo de memoria.
Argon2	2015	Ganador del Password Hashing Competition.

4. Funciones hash y derivación de claves (KDF)

Argon2 es el “más moderno” y se considera el mejor en 2025, pero **PBKDF2 sigue siendo perfectamente válido y es el recomendado por NIST para entornos**

4.4 ¿Qué KDF usa Clef?

Clef usa **PBKDF2-HMAC-SHA256 con 230.000 iteraciones**.

Vamos a desmenuzar ese nombre:

- **PBKDF2** = Password-Based Key Derivation Function 2. La función en sí.
- **HMAC** = Hash-based Message Authentication Code. Es la primitiva interna que PBKDF2 invoca repetidamente.
- **SHA256** = el algoritmo hash subyacente.
- **230.000 iteraciones** = el número de veces que se aplica HMAC-SHA256 internamente.

Esto significa que cada vez que el usuario introduce su contraseña maestra, el dispositivo tiene que calcular **230.000 HMAC-SHA256 encadenados**. En un teléfono moderno tarda alrededor de **400 milisegundos**. Para el usuario es imperceptible (medio segundo de “cargando”). Para un atacante que quiera probar 10 millones de contraseñas, esos 400 ms se multiplican por 10 millones = **4.000.000 segundos = ~46 días** de cómputo continuo, y eso *por cada usuario*.

¿Por qué exactamente 230.000?

OWASP en 2023 actualizó la recomendación mínima a **600.000 iteraciones** para PBKDF2-SHA256 en sistemas de servidor. Para sistemas móviles, donde el dispositivo del usuario tiene que ejecutar el KDF en cada login, la cifra suele bajar para no penalizar la experiencia. 230.000 está por encima del antiguo mínimo de 210.000 y por debajo del nuevo recomendado de 600.000. Es un compromiso razonable para Android, donde un PBKDF2 muy alto castiga sobre todo a teléfonos antiguos.

4.5 ¿Qué obtiene PBKDF2?

PBKDF2 toma como entradas: - Una contraseña (la del usuario). - Un salt (un valor aleatorio único — sección 5). - Un número de iteraciones. - Una longitud de salida deseada.

Y produce una **clave criptográfica de tamaño fijo**, en el caso de Clef de 256 bits, lista para usar como clave AES.

Contraseña + Salt + 230.000 iteraciones HMAC-SHA256



Clave AES-256 (32 bytes)

Esa clave derivada en Clef se llama **KEK** (Key Encryption Key) y es la que se usa para envolver la DEK que cifra la bóveda. La DEK y la KEK las veremos en la sección 8.

5. La salt: qué es y por qué existe

La salt es uno de esos conceptos que parecen triviales pero son responsables de detener clases enteras de ataques. Vamos a verlo despacio.

5.1 El ataque que la salt previene

Imaginemos que un servicio guarda las contraseñas de sus usuarios como SHA-256 directo, sin salt:

5. La salt: qué es y por qué existe

Usuario	Hash de la contraseña
alice@x.com	ef92b...bc3
bob@y.com	ef92b...bc3
eve@z.com	8a7c2...91d

Ya hay una pista enorme: alice y bob tienen el **mismo hash**, lo que revela que ambos usaron la misma contraseña sin que el atacante haya hecho nada todavía.

Y hay más. Como SHA-256 es determinista y rápido, un atacante con tiempo previo puede precalcular una **rainbow table** masiva:

```
SHA-256("password") = 5e884...d8
SHA-256("123456")   = 8d969...96
SHA-256("qwerty")   = 65e84...d3
SHA-256("admin")    = 8c692...07
... [millones más] ...
```

Cuando obtiene la base de datos filtrada, busca cada hash en su tabla. Si el hash de Alice está en la tabla, conoce la contraseña en milisegundos.

5.2 La idea de la salt

La **salt** es un valor **aleatorio**, **único por usuario** y **público** (no se mantiene en secreto) que se mezcla con la contraseña antes de hashearla:

```
hash_guardado = PBKDF2("contraseña" + salt_único, iteraciones)
```

Vamos a ver qué pasa ahora con el mismo escenario:

Usuario	Salt	Hash
alice@x.com	a1b2c3...	1f8a2...d4
bob@y.com	9f4e8b...	7c3a1...8b
eve@z.com	23bcfe...	92e5b...0c

Aunque alice y bob usen la misma contraseña, los hashes son **completamente distintos** porque la salt mezclada es distinta. El atacante ya no puede saber a simple vista que comparten contraseña.

Y lo más importante: las **rainbow tables son inservibles**. Para que el atacante atacara a alice con una rainbow table, tendría que precalcular una tabla *específica para la salt de alice*. Y después otra para la de bob. Y otra para eve. Y otra para cada uno de los millones de usuarios. El coste es absurdo.

5.3 ¿La salt tiene que ser secreta?

No. La salt se guarda *junto* al hash en la base de datos, en claro. Esto suele sorprender al principio. La razón es que la seguridad de la salt no depende de que sea secreta — depende de que sea **aleatoria y única**. Lo que la salt evita es que el atacante pueda *amortizar* el coste de calcular hashes entre múltiples usuarios o pre-calcularlos antes del ataque.

Si la salt es de 32 bytes aleatorios (como en Clef), hay 2^{256} valores posibles. El atacante podría conocer la salt de alice y aun así no podría tener una tabla precalculada para ella, porque tendría que haberla anticipado entre 2^{256} posibilidades.

5.4 La salt en Clef

Cuando un usuario de Clef se registra:

```
byte[] salt = CryptoUtils.generateSalt(); // 32 bytes aleatorios
byte[] kek = CryptoUtils.deriveKey(password, salt); // PBKDF2(pwd, salt, 230k)
```

La salt se genera con `SecureRandom`, una fuente de entropía criptográficamente segura, **una sola vez por usuario** y nunca cambia (a menos que se borre la cuenta). Se guarda en Firestore en el documento del usuario, en claro, junto a las cajas A y B.

6. Arquitecturas de seguridad en gestores de contraseñas

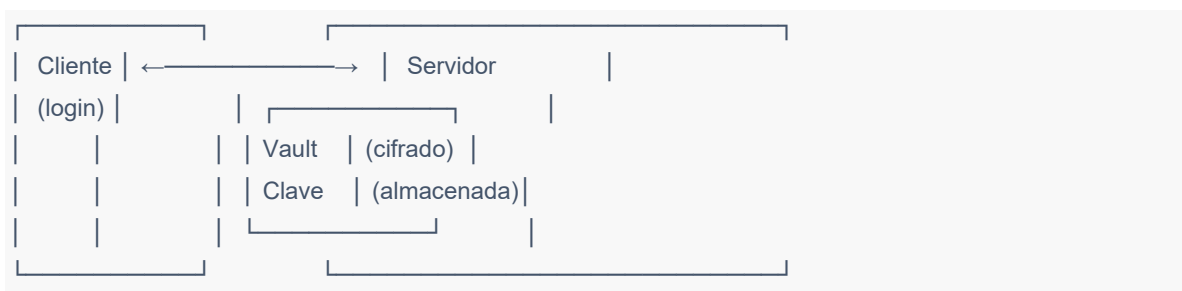
Esto significa que aunque dos usuarios distintos eligieran la misma contraseña maestra “Madrid2024!”, sus salts serían distintas, sus KEKs serían distintas, sus Cajas A serían distintas, y sus DEKs (las claves de los datos) serían también distintas. **La filtración de uno no compromete al otro de ninguna manera.**

6. Arquitecturas de seguridad en gestores de contraseñas

Cuando se diseña un gestor de contraseñas hay que tomar una decisión arquitectónica grande antes de tocar una sola línea de código: **¿quién tiene la capacidad de leer los datos del usuario?** Esa pregunta separa tres modelos muy distintos.

6.1 Modelo cliente-servidor tradicional

El más sencillo y el peor desde el punto de vista de seguridad. El servidor guarda los datos del usuario y aplica algún tipo de cifrado, pero **el servidor también guarda las claves**. Cuando el usuario se autentica, el servidor descifra y le devuelve los datos.

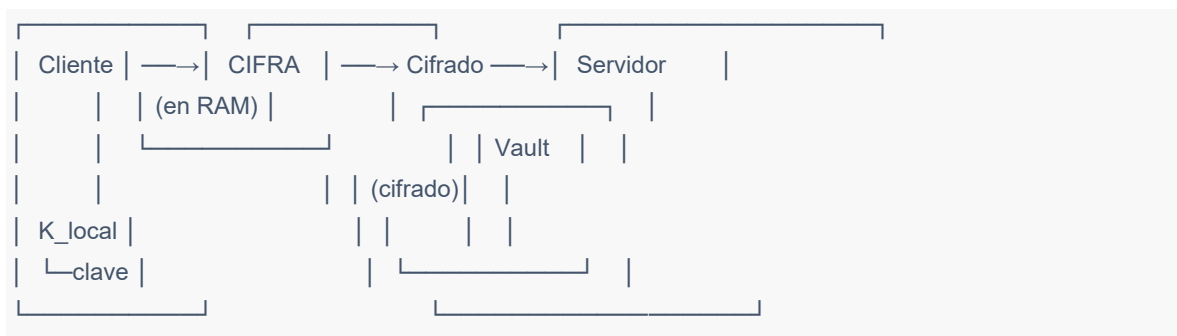


Modelos así suelen anunciarse como “cifrados”, lo cual es técnicamente cierto pero engañoso: si el atacante (o el propio proveedor, o un empleado deshonesto, o la policía con una orden) accede al servidor, accede a los datos. La cadena de confianza depende íntegramente del proveedor.

Ejemplos históricos: muchas filtraciones masivas de servicios “seguros” se dieron por este modelo. El caso paradigmático fue **LastPass en 2022**: un atacante robó copias completas de las bóvedas de millones de usuarios. Aunque LastPass las almacenaba cifradas, partes del esquema dependían del servidor y la calidad del cifrado en datos individuales como URLs era cuestionable. Las consecuencias se extendieron durante años.

6.2 Modelo end-to-end encryption (E2EE)

El cifrado y el descifrado ocurren **solo en el cliente**. El servidor recibe ya datos cifrados y los almacena. El servidor no tiene la clave.



Esto ya es mucho mejor que el anterior. Pero no es lo mismo que Zero-Knowledge — la diferencia es sutil pero importante.

6.3 Modelo Zero-Knowledge (ZK)

Es un superconjunto de E2EE. Además de que el cifrado/descifrado sea en cliente, **la propia contraseña maestra (o derivado) nunca llega al servidor en ninguna forma**.

6. Arquitecturas de seguridad en gestores de contraseñas

EL SERVIDOR NO SABE:

- La contraseña maestra
- Las claves de cifrado
- El contenido de la bóveda

EL SERVIDOR SOLO TIENE:

- Salt (público, aleatorio)
- DEK cifrada con KEK_master (Caja A)
- DEK cifrada con KEK_PUK (Caja B)
- Bóveda cifrada con DEK

La diferencia fina con E2EE puro es que en algunos sistemas E2EE el servidor sí participa en la autenticación (por ejemplo, recibiendo un hash de la contraseña). En ZK estricto, el servidor **nunca recibe la contraseña ni nada derivado de ella**. La autenticación de identidad la hace Firebase con email/contraseña Google, pero **esa autenticación es independiente del cifrado de los datos**: Firebase autentica al usuario para que pueda *acceder* a su documento, pero los datos dentro del documento están cifrados con una clave que Firebase nunca ha visto.

6.4 Comparativa

Modelo	Cifrado en	Clave guardada en	Si el servidor cae	Si pierdes contraseña
Cliente-servidor	Servidor	Servidor	Datos comprometidos	El servidor te ayuda
E2EE	Cliente	Cliente (a veces servidor también)	Depende	Variable
Zero-Knowledge	Cliente	Solo cliente	Datos seguros	Nadie puede ayudarte

6.5 Otros modelos relevantes

- **HSM (Hardware Security Module):** las claves se guardan en hardware especializado nunca accesible por software. Es lo que usa Android Keystore en Clef para la clave biométrica.
- **Threshold cryptography / Shamir's Secret Sharing:** la clave se reparte entre N partes, y se necesitan al menos K para reconstruirla. Sirve para casos corporativos donde no quieres que ninguna persona tenga la clave entera.
- **Homomorphic encryption:** permite hacer operaciones sobre datos cifrados sin descifrarlos. Investigación punta, todavía demasiado lento para uso general.

7. Por qué Clef es Zero-Knowledge

7.1 La razón filosófica

El argumento es simple: **la mejor defensa contra una filtración del servidor es que el servidor no tenga nada útil que filtrar.**

Cualquier modelo donde el proveedor guarde las claves o derivados secretos requiere confiar en:

- Que el proveedor no se equivoque nunca con sus configuraciones de seguridad.
- Que ninguno de sus empleados sea malicioso.
- Que ningún gobierno le obligue a entregar los datos.
- Que ningún atacante consiga acceso a sus servidores.

7. Por qué Clef es Zero-Knowledge

Cada uno de esos cuatro puntos ha fallado en gestores de contraseñas reales en los últimos diez años. Zero-Knowledge elimina esa cadena de confianza: **da igual** que el servidor sea malicioso, esté comprometido o esté bajo coacción legal, porque no tiene los datos ni puede obtenerlos.

7.2 La razón técnica concreta en Clef

Firebase Firestore es un servicio de Google. Es robusto, escalable y tiene reglas de seguridad excelentes. Pero Google es una empresa estadounidense sujeta a la ley estadounidense, y el **CLOUD Act de 2018** permite al gobierno de EEUU obligar a empresas estadounidenses a entregar datos almacenados en sus servidores **incluso si esos datos están físicamente en otro país**.

Si los datos de Clef se guardaran en claro en Firestore, Google podría verse legalmente obligado a entregárselos a un gobierno. Con la arquitectura ZK, lo único que Google podría entregar son los blobs cifrados, que sin la contraseña maestra del usuario son ruido aleatorio.

Esto es importante para usuarios en contextos sensibles (periodistas, activistas, profesionales con datos confidenciales) y no debería serlo menos para el usuario corriente.

7.3 Ventajas

- **Resistencia a brechas del servidor:** ya analizado. Lo más importante.
- **Independencia del proveedor:** Clef podría migrar mañana de Firestore a otro backend sin que ningún dato del usuario quede expuesto durante la transición. No hay claves en el servidor que migrar.
- **Resistencia a coacción legal:** el proveedor no puede entregar lo que no tiene.

- **Resistencia a empleados deshonestos:** ni siquiera el equipo del proyecto puede leer los datos de los usuarios, aunque quisiera.
- **Aislamiento entre usuarios:** con la arquitectura por UID, el compromiso de la cuenta de un usuario no afecta a los demás.

7.4 Desventajas (y son reales)

Hay que ser honesto. Zero-Knowledge tiene costes que no se pueden ocultar:

- **No hay recuperación de cuenta clásica:** si el usuario olvida su contraseña maestra y pierde su PUK, *los datos están perdidos para siempre*. Ni el equipo de Clef ni nadie puede recuperarlos. Esto es así por diseño y es la consecuencia inevitable de no tener las claves en el servidor.
- **Toda la pesada criptografía corre en el cliente:** PBKDF2 con 230.000 iteraciones ocurre en el teléfono del usuario, no en un servidor potente. Esto da ~400 ms de latencia en cada login.
- **El cliente tiene que ser confiable:** si el atacante compromete el dispositivo del usuario (malware, root, dispositivo robado y desbloqueado), puede acceder a los datos en RAM. ZK protege el servidor, no el cliente.
- **No se puede ofrecer “sincronización entre dispositivos automática” sin re-login:** cada dispositivo nuevo necesita derivar la KEK desde cero introduciendo la contraseña.
- **Búsqueda y operaciones avanzadas son imposibles desde el servidor:** no se puede ofrecer, por ejemplo, “encuentra todas las contraseñas que contienen mi email” desde una API. Todo se procesa cliente.

8. Algoritmos concretos implementados en Clef

Esta sección entra al detalle técnico. Es la parte que el tribunal va a querer ver con código.

8.1 AES-256-GCM (cifrado simétrico autenticado)

AES (Advanced Encryption Standard) es el estándar de cifrado simétrico desde 2001, sucesor de DES. Lo eligió NIST tras una competición pública entre 15 algoritmos. AES tiene tres tamaños de clave: 128, 192 y 256 bits. Clef usa el más fuerte: **AES-256**.

Modo de operación: GCM (Galois/Counter Mode). Esta es una decisión importante.

AES en sí mismo solo cifra **bloques de 128 bits**. Para cifrar mensajes más largos hace falta un “modo de operación” que defina cómo encadenar los bloques. Existen muchos modos:

Modo	Confidencialidad	Integridad	Notas
ECB	✓	X	Inseguro. Patrones del texto se ven en el cifrado.
CBC	✓	X	Necesita HMAC aparte para integridad. Vulnerable a padding oracle.
CTR	✓	X	Igual que CBC en cuanto a integridad.
GCM	✓	✓	AEAD: cifra y autentica en una sola operación.

GCM es **AEAD** (Authenticated Encryption with Associated Data). Eso significa que produce no solo el texto cifrado, sino también un **tag de autenticación** de 128 bits. Si alguien modifica un solo bit del cifrado, el tag no cuadra al descifrar y la operación falla con `AEADBadTagException` antes de devolver datos.

Esto es crucial porque sin AEAD, un atacante que tenga acceso a la base de datos cifrada **podría modificar bytes** sin que Clef se diera cuenta, lo que abre ataques de manipulación.

El código en Clef (`CryptoUtils.encrypt`):

```
byte[] iv = generateRandomBytes(GCM_IV_BYTES); // 12 bytes aleatorios

Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
cipher.init(Cipher.ENCRYPT_MODE,
    new SecretKeySpec(keyBytes, "AES"),
    new GCMParameterSpec(128, iv)); // tag de 128 bits

byte[] ciphertext = cipher.doFinal(plaintext);

// Resultado: [IV (12B)][ciphertext + tag (16B)]
byte[] combined = new byte[iv.length + ciphertext.length];
System.arraycopy(iv, 0, combined, 0, iv.length);
System.arraycopy(ciphertext, 0, combined, iv.length, ciphertext.length);
```

El IV (Initialization Vector) es un valor aleatorio de 96 bits. Cumple un papel similar al salt: hace que cada operación de cifrado sea diferente aunque la clave y el contenido sean los mismos. **Crítico:** en GCM, **reutilizar un IV con la misma clave rompe la seguridad por completo**. Por eso Clef lo regenera con `SecureRandom` cada vez que cifra algo.

8.2 PBKDF2-HMAC-SHA256 (derivación de clave)

Ya explicado en la sección 4. El código:

```
public static byte[] deriveKey(char[] password, byte[] salt) throws Exception {
    PBEKeySpec spec = new PBEKeySpec(
        password,
        salt,
        230_000,    // iteraciones
        256         // longitud de salida (bits)
    );
    return SecretKeyFactory
        .getInstance("PBKDF2WithHmacSHA256")
        .generateSecret(spec)
        .getEncoded();
}
```

Tres detalles importantes:

- `char[]` y no `String`: en Java, los `String` son inmutables y no se pueden borrar de memoria. Las contraseñas viajan como `char[]` para poder zerizarlas con `Arrays.fill(password, '\0')` inmediatamente después de derivar la clave.
- `spec.clearPassword()` se llama en el `finally` para limpiar la copia interna que `PBEKeySpec` mantiene.
- **El resultado son 32 bytes = 256 bits**, listos como clave AES.

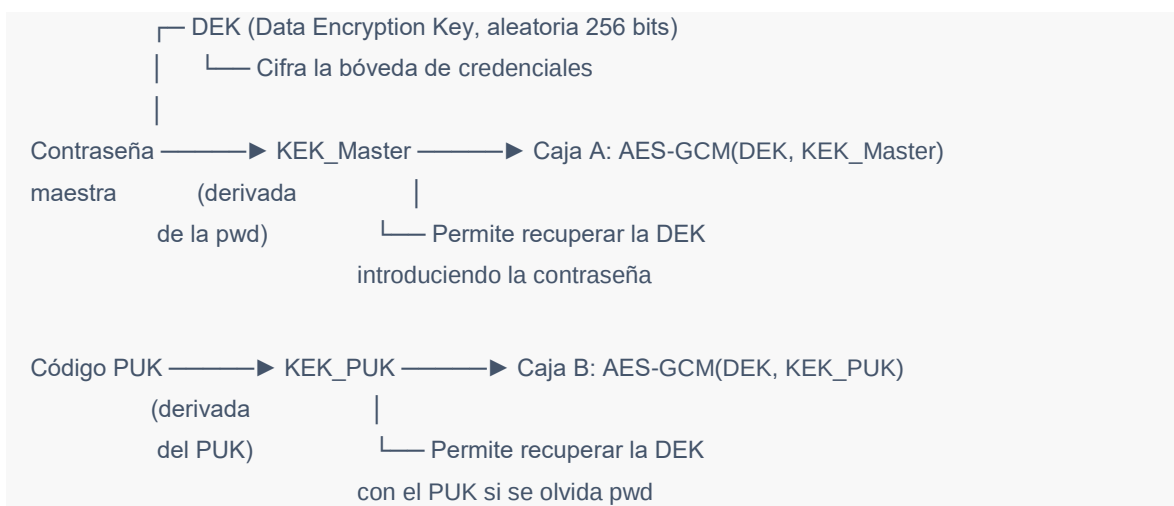
8.3 SecureRandom (generación de aleatoriedad)

Toda la entropía de Clef (salts, IVs, DEKs, PUKs) viene de `java.security.SecureRandom`, que en Android se respalda en `/dev/urandom` y en hardware del SoC cuando está disponible (la mayoría de los chips Qualcomm/Samsung modernos tienen un TRNG físico).

`SecureRandom` es radicalmente diferente de `Random` o `Math.random()`. Estos últimos son **predecibles**: si conoces la semilla, conoces toda la secuencia. Usar `Random` para generar claves criptográficas ha sido la causa de varios bugs históricos catastróficos (como el caso de Debian OpenSSL en 2008, donde un commit mal hecho hizo que las claves SSH generadas durante 2 años fueran predecibles).

8.4 Key Wrapping: la jerarquía de claves de Clef

Aquí está el corazón de la arquitectura. Clef no cifra los datos del usuario directamente con la clave derivada de la contraseña. Usa un esquema de **dos niveles**:



¿Por qué esta complicación? Tres razones:

1. **Cambio de contraseña sin re-cifrar la bóveda.** Si el usuario quiere cambiar su contraseña maestra, no hay que descifrar y volver a cifrar todas las credenciales. Solo se descifra la DEK con la KEK_Master vieja, se deriva una KEK_Master_nueva con la nueva contraseña, y se vuelve a cifrar la DEK con KEK_Master_nueva. La bóveda en sí no se toca. Si tienes 500 credenciales, esto es la diferencia entre cifrar 500 cosas o 1 sola.
2. **Múltiples vías de acceso a la misma DEK.** Tanto la contraseña maestra como el PUK abren la misma DEK por caminos distintos. Si pierdes una vía, tienes la otra. Pero ambas convergen en la misma DEK, así que la bóveda no se duplica.
3. **Patrón estándar en la industria.** Es el mismo modelo que usan 1Password, Bitwarden y Proton Pass. No es un invento de Clef; es la forma probada de hacer esto bien.

8.5 Android Keystore (TEE para biometría)

Cuando el usuario activa el desbloqueo biométrico, Clef genera una clave AES-256 dentro del **Android Keystore**. La Keystore es una API de Android que delega la custodia de claves al **TEE** (Trusted Execution Environment) del SoC, un entorno aislado del sistema operativo.

Característica fundamental: la clave **nunca sale del TEE**. Cuando Clef quiere cifrar la DEK con esa clave, no obtiene la clave; obtiene un objeto *Cipher* ya inicializado por el TEE, lo usa, y la operación criptográfica ocurre dentro del hardware seguro.

Configuración usada:

```
new KeyGenParameterSpec.Builder(  
    KEY_ALIAS,  
    PURPOSE_ENCRYPT | PURPOSE_DECRYPT)  
    .setBlockModes(BLOCK_MODE_GCM)  
    .setEncryptionPaddings(ENCRYPTION_PADDING_NONE)  
    .setKeySize(256)  
    .setUserAuthenticationRequired(true)           // pide huella/cara para usar  
    .setInvalidatedByBiometricEnrollment(true)     // se invalida si añaden huella nueva  
    .build();
```

`setInvalidatedByBiometricEnrollment(true)` es la pieza clave: si alguien añade una huella nueva al dispositivo (porque te lo robaron y se enrolaron), la clave **se autodestruye** automáticamente. La DEK biométrica deja de poder descifrarse y la única vía es la contraseña maestra.

9. Flujo criptográfico completo de la aplicación

Vamos a recorrer Clef paso a paso. El objetivo es que cualquier miembro del tribunal pueda seguir qué hace exactamente la app cuando el usuario interactúa con ella.

9.1 Registro (primer uso, `KeyManager.register()`)

1. El usuario introduce su contraseña maestra (mínimo 8 caracteres).
2. `SecureRandom` genera:
 - Salt aleatorio de 32 bytes.
 - DEK aleatoria de 32 bytes.
 - PUK aleatorio de 16 bytes (que se formatea como 32 chars hex con guiones).
3. `PBKDF2(contraseña, salt, 230.000 iter) → KEK_Master (256 bits)`
4. `PBKDF2(PUK, salt, 230.000 iter) → KEK_PUK (256 bits)`
5. `AES-GCM-Cifra(DEK, KEK_Master) → Caja A`
6. `AES-GCM-Cifra(DEK, KEK_PUK) → Caja B`
7. `AES-GCM-Cifra(JSON("[]"), DEK) → Bóveda inicial vacía cifrada`

9. Flujo criptográfico completo de la aplicación

8. Sube a Firestore (en una sola escritura atómica):

```
{ salt, cajaA, cajaB, vault, version: 0 }
```

9. Muestra el PUK al usuario UNA VEZ. NUNCA SE PERSISTE.

10. Sobrescribe en RAM con ceros: contraseña, KEK_Master, KEK_PUK, PUK.

11. Retiene en sesión: solo la DEK (para cifrar credenciales nuevas).

Tras este flujo, **Firestore contiene 4 blobs cifrados y un salt**. Nada más. La contraseña, las claves derivadas y el PUK ya no existen en ningún sitio (excepto en la cabeza del usuario y en el papel donde apuntó el PUK).

9.2 Login diario (*KeyManager.login()*)

1. La app intenta cargar caché local (vault_<uid>.enc + claves cacheadas en EncryptedSharedPreferences).

└─ Si no existe (dispositivo nuevo): descarga de Firestore.

2. El usuario introduce la contraseña maestra.

3. PBKDF2(contraseña, salt_almacenado, 230k) → KEK_Master
(~400 ms en hilo de fondo)

4. AES-GCM-Descifra(Caja A, KEK_Master) → DEK

└─ Contraseña incorrecta: AEADBadTagException → error controlado
| y BruteForceGuard registra el intento fallido.

└─ Contraseña correcta: continúa.

5. AES-GCM-Descifra(Bóveda, DEK) → JSON → Vault cargado en RAM

6. Sobrescribe en RAM: contraseña, KEK_Master.

La DEK queda en SessionManager hasta que se bloquee la app.

Lo que un atacante podría observar pasivamente en este flujo: - Que se hace una operación de PBKDF2 en el cliente. (Sin valor.) - Que se hace una llamada a Firestore para leer el documento del usuario. (Sin valor: el contenido viaja por TLS y, además, está cifrado.)

9.3 Guardar una credencial nueva (*AddItemDialog*)

1. El usuario rellena título, usuario, contraseña, etc.
2. Se añade el nuevo Credential al Vault en RAM (`SessionManager.getVault()`).
3. AES-GCM-Cifra(JSON(Vault completo), DEK) → Bóveda actualizada cifrada
4. Se escribe la bóveda cifrada en disco local (`vault_<uid>.enc`).
5. Si la sincronización está activada:
 - `FirebaseManager.uploadVaultVersioned(boveda_cifrada, version_actual, version+1)`
 - Optimistic concurrency control con transacción Firestore.
 - Si la versión del servidor cambió mientras tanto (otro dispositivo guardó algo) → `CONFLICT_ERROR` → retry automático con merge.
 - Si va bien → `version++` y `SessionManager` refleja la nueva.

9.4 Recuperación con PUK (*KeyManager.recoverWithPuk()*)

1. El usuario olvida la contraseña maestra. Pulsa "Usar PUK".
2. Se descarga de Firestore: { salt, cajaB, vault }.
3. El usuario introduce el PUK.
4. `PBKDF2(PUK, salt, 230k) → KEK_PUK`
5. `AES-GCM-Descifra(Caja B, KEK_PUK) → DEK` ← ¡La misma DEK de siempre!
6. `AES-GCM-Descifra(Vault, DEK) → Bóveda`
7. El usuario elige una contraseña maestra nueva.
8. `PBKDF2(nueva_contraseña, salt, 230k) → KEK_Master_nueva`
9. `AES-GCM-Cifra(DEK, KEK_Master_nueva) → Caja A nueva`
10. `SecureRandom` genera nuevo PUK (16 bytes aleatorios).
11. `PBKDF2(nuevo_PUK, salt, 230k) → KEK_PUK_nueva`
12. `AES-GCM-Cifra(DEK, KEK_PUK_nueva) → Caja B nueva`

9. Flujo criptográfico completo de la aplicación

13. ESCRITURA ATÓMICA en Firestore: nueva Caja A + nueva Caja B.

El PUK viejo queda criptográficamente invalidado: la Caja B que abría ya no existe; ha sido sobrescrita por la nueva.

14. Se muestra el nuevo PUK al usuario UNA VEZ.

El detalle elegante: el PUK no se “marca como usado” ni se compara con una lista. Queda invalidado **porque la Caja B que descifraba ya no existe físicamente**. Es invalidación criptográfica, no lógica.

9.5 Auto-Lock

La app va a background → `ProcessLifecycleOwner.onStop()`

→ `SessionManager.startLockTimer()`

└─ Espera el tiempo configurado (1, 5, 15, 30 min).

└─ Si el usuario no vuelve antes:

`lock()`:

`Arrays.fill(dek, 0x00)` ← borra DEK

`vault = null` ← borra vault

dispara `onLock()` listener

→ siguiente apertura: `UnlockActivity`

El bloqueo **no es lógico**; es físico: la DEK se sobrescribe con ceros en el array de bytes. Aunque alguien obtuviera un volcado de memoria del proceso justo después del bloqueo, encontraría ceros donde antes estaba la clave.

9.6 Biometría (cuando está activada)

ACTIVAR BIOMETRÍA:

1. La app pide al Android Keystore: "genera una clave AES-256 para esta app".

Configuración: GCM, sin padding, requiere autenticación de usuario, invalidable al añadir huella nueva.

2. Android Keystore genera la clave dentro del TEE. La clave nunca sale.

3. La app pide al usuario que se autentique con huella.

4. Tras autenticación, el TEE expone un Cipher inicializado.

5. `Cipher.doFinal(DEK)` → `DEK_cifrada_por_keystore`.

6. Se guarda `DEK_cifrada_por_keystore` + IV en `EncryptedSharedPreferences`

(que añade otra capa, esta vez con clave gestionada por Android).

DESBLOQUEO POR BIOMETRÍA:

1. Lee DEK_cifrada_por_keystore + IV de EncryptedSharedPreferences.
2. Pide al usuario huella/cara.
3. Tras autenticación, el TEE expone un Cipher inicializado para descifrar.
4. Cipher.doFinal(DEK_cifrada_por_keystore) → DEK en claro en RAM.
5. Sigue el flujo normal de login: descifrar la bóveda con la DEK, etc.

la clave AES del Keystore **nunca toca el espacio de usuario**. Si alguien hace root del teléfono y vuelca toda la RAM del proceso de Clef, encontrará la DEK *si Clef está desbloqueado en ese momento*, pero no encontrará la clave del Keystore que cifró la DEK. Esa clave vive en otro mundo (el TEE).

10. Modelo de amenazas y mitigaciones

Cualquier sistema de seguridad serio se evalúa contra un **modelo de amenazas**: un catálogo de quién puede atacarlo, con qué capacidades, y qué hace el sistema en cada caso. Esta sección es exactamente eso.

10.1 Ataque de fuerza bruta online

Escenario: un atacante intenta adivinar la contraseña maestra probando muchas combinaciones desde la app de Clef en su propio teléfono o desde un script automatizado.

Mitigación en Clef: - `BruteForceGuard` registra cada intento fallido en `EncryptedSharedPreferences` (no en `SharedPreferences` sin cifrar — un atacante con acceso ADB no puede resetear el contador). - Bloqueo progresivo: 10 s tras el primer fallo, 30 s tras el segundo, 60 s, 120 s, 240 s. - Tras 5 fallos consecutivos, se redirige al usuario al flujo de recuperación con PUK. - Los contadores son **por UID**: bloquear al atacante en una cuenta no afecta a otras cuentas en el mismo dispositivo.

Coste real para el atacante: probar 1.000 contraseñas tarda como mínimo $5 \times 10s$ + $tiempo_PBKDF2 = 50$ segundos por bloque de 5, más 4 minutos de bloqueo, más PBKDF2 (~400 ms). Es **inviable** intentar siquiera un diccionario de las top 10.000 contraseñas más comunes desde la app oficial.

10.2 Ataque de fuerza bruta offline

Escenario: el atacante consigue copiar la base de datos cifrada (Firestore comprometido, o bóveda local de un teléfono robado) y la ataca en sus propias máquinas, sin pasar por la lógica de bloqueo de Clef.

Por qué es más peligroso que el online: sin la lógica de bloqueo, el atacante puede probar tan rápido como su hardware le permita.

Mitigación en Clef: - **PBKDF2 con 230.000 iteraciones** convierte cada intento en ~400 ms en un teléfono o ~50-100 ms en una GPU dedicada (las GPUs son más rápidas en PBKDF2-HMAC-SHA256, pero solo en un orden de magnitud, no en muchos como con SHA-256 directo). - **Salt único de 32 bytes:** imposibilita rainbow tables y obliga al atacante a hacer el ataque dirigido a un solo usuario, sin amortizar coste entre víctimas. - Estimación con una RTX 4090: ~3-5 millones de PBKDF2-HMAC-SHA256 230k por segundo.

Coste para el atacante con una contraseña típica:

Contraseña usuario	Espacio de búsqueda	Tiempo estimado RTX 4090
<i>123456</i>	Top 100	<1 segundo
<i>Madrid2024</i>	Top 100k	~30 segundos
<i>Mi-perro-Toby-2019!</i>	Diccionario contextual	Horas a días
8 chars aleatorios	6×10^{15}	~4 años
12 chars aleatorios	5×10^{23}	edad del universo

Conclusión honesta: PBKDF2 protege bien, pero **no compensa una contraseña terrible**. Por eso la app muestra un indicador de fortaleza al crear la contraseña maestra y exige mínimo 8 caracteres. Un usuario que ponga `password` no está protegido frente a un atacante decidido aunque la base de datos use PBKDF2 con 10 millones de iteraciones.

10.3 Ataque Man-in-the-Middle (MitM)

Escenario: un atacante intercepta la comunicación entre la app y Firestore, intentando leer o modificar el tráfico.

Mitigación en Clef: - **Todo el tráfico va por HTTPS / TLS** (Firebase SDK no permite HTTP en claro). - `network_security_config.xml` declara explícitamente `cleartextTrafficPermitted="false"`. Aunque alguien modificara el SDK para hacer HTTP, Android lo bloquearía a nivel del sistema operativo. - **Y aunque el atacante consiguiera bajar el TLS**, lo que vería son los blobs ya cifrados con AES-GCM. La comunicación es: cliente envía blobs cifrados, servidor los almacena. El atacante no obtiene nada útil aunque grabe el tráfico completo. - **Integridad GCM:** si el atacante intentara modificar blobs en tránsito (tipo “voy a corromper la Caja A para forzar reset”), el descifrado fallaría con `AEADBadTagException` y la app no procesaría datos manipulados.

10.4 Robo del dispositivo

Escenario: alguien roba el teléfono del usuario.

Mitigación en Clef: - **Auto-lock:** tras 5 minutos por defecto en background, la sesión se invalida y la DEK se sobrescribe con ceros. El atacante necesita la contraseña maestra para entrar. - **Almacenamiento cifrado:** incluso accediendo al filesystem del teléfono (via ADB en un teléfono rooteado o por extracción forense), `vault_<uid>.enc` está cifrado con AES-GCM y no se puede leer sin la DEK. -

10. Modelo de amenazas y mitigaciones

`allowBackup="false"`: Android no exporta los datos de la app en backups automáticos a Google Drive. - **Aislamiento por UID**: si el atacante consigue entrar como otro usuario (creando una cuenta nueva), no puede leer el vault del usuario robado porque está nombrado con el UID viejo. - **Biometría con `setInvalidatedByBiometricEnrollment(true)`**: si el atacante registra su propia huella en el dispositivo robado, la clave biométrica de Clef se invalida. Solo queda la vía de la contraseña maestra.

Limitación honesta: si el dispositivo está desbloqueado y Clef tiene sesión activa (DEK en RAM), un atacante con privilegios podría volcar la memoria. Esto es una limitación estructural de cualquier app que no usa SGX o equivalentes — para mitigarlo sería necesario que Clef solo desbloqueara la DEK durante operaciones puntuales y la zerizara entre operaciones, lo cual penaliza UX brutalmente.

10.5 Phishing

Escenario: un atacante crea una app falsa que parece Clef y se la instala al usuario, capturando la contraseña maestra al introducirla.

Mitigación parcial en Clef: - Las apps de Google Play tienen verificación de firma por el Play Store. Un usuario que solo instala desde Play Store está protegido. - `FLAG_SECURE` en todas las activities impide que apps maliciosas hagan capturas de pantalla automáticas durante el uso de Clef (esto sí es una mitigación específica que la mayoría de apps no implementan). - `usesCleartextTraffic="false"` impide que un proxy malicioso aceptara HTTP en claro.

Limitación: ninguna app puede protegerse al 100% de un usuario que se descarga e instala una APK falsa fuera de Play Store. La defensa es educación + Play Protect.

10.6 Manipulación de la base de datos

Escenario: un atacante con acceso al servidor modifica los blobs cifrados (no para leerlos, sino para corromperlos o sustituirlos por otros con intención maliciosa).

Mitigación en Clef: - **AES-GCM es AEAD:** si el atacante manipula un solo bit, el descifrado falla con `AEADBadTagException` y la app no procesa el dato manipulado. - **Reglas de Firestore:** cada usuario solo puede escribir en su propio documento. Un atacante autenticado como otro usuario no puede modificar el del usuario víctima.

```
match /users/{userId}/{document=**} {  
  allow read, write: if request.auth != null  
    && request.auth.uid == userId;  
}
```

10.7 Replay attacks

Escenario: un atacante captura una sesión válida (cifrada con TLS) y la reproduce más tarde para hacerse pasar por el usuario.

Mitigación: - TLS 1.3 (que es lo que usa Firebase SDK por defecto) tiene defensas contra replay a nivel de protocolo. - Firebase Auth gestiona tokens de sesión con expiración corta (1 hora) y refresh tokens.

10.8 Side-channel attacks

Escenario: ataques que no atacan el algoritmo en sí, sino fugas de información del sistema (timing, consumo eléctrico, etc.).

Mitigación: - AES-NI en hardware ejecuta AES en tiempo constante (sin variaciones por contenido). Las CPUs modernas casi siempre lo tienen. - Las comparaciones de tags GCM se hacen byte a byte completas, sin short-circuit, evitando timing attacks en la verificación.

11. El caso crítico: “¿qué pasa si roban Firestore?”

Limitación honesta: los ataques side-channel sofisticados (como diferenciales de potencia con osciloscopio) requieren acceso físico al chip y van más allá del modelo de amenazas razonable de una app móvil.

10.9 Compromiso del cliente (root, malware)

Escenario: el dispositivo del usuario tiene root o malware activo.

Mitigación parcial: - `RootDetector` detecta indicios de root (binarios `su`, apps de root conocidas, `ro.debuggable=1`, etc.) y bloquea el acceso a la app si hay evidencia clara.

- `ProGuard/R8` ofusca el código en release, dificultando ingeniería inversa.

Limitación honesta: un atacante con root y conocimientos técnicos puede vencer cualquier detección de root. Es un equilibrio: hace falta cierto esfuerzo y no se le pone fácil, pero no es invencible. La defensa real contra malware es que el usuario no instale apps fuera de Play Store y mantenga el sistema actualizado.

11. El caso crítico: “¿qué pasa si roban Firestore?”

Esta es probablemente la pregunta que el tribunal va a hacer literal. Vamos a darle una respuesta exhaustiva.

Escenario: un atacante consigue una copia completa de la base de datos de Firestore. Esto puede ocurrir por: - Una brecha en Google (improbable pero ha pasado en otras compañías). - Una mala configuración de las reglas de Firestore (un bug de Clef hipotético). - Coacción legal (un gobierno obliga a Google a entregar los datos). - Un empleado deshonesto de Google con acceso al sistema.

Lo que el atacante obtiene para cada usuario:



Lo que el atacante PUEDE hacer:

1. **Identificar al usuario por su email** (porque Firebase Auth guarda el email del usuario en otro documento). Esto sí es información personal — pero es algo que Google ya tiene de todas formas y no es información del gestor en sí.
2. **Saber con qué frecuencia el usuario actualiza su bóveda** (campo `version`).
3. **Saber el tamaño aproximado de su bóveda** (longitud del campo `vault`). Esto puede dar una pista de si tiene 5 o 500 contraseñas guardadas. No es información que comprometa por sí sola.

Lo que el atacante NO PUEDE hacer:

1. **Leer ninguna credencial.** La bóveda está cifrada con la DEK, que está cifrada con la KEK_Master, que se deriva de una contraseña que el atacante no tiene.
2. **Saber qué servicios usa el usuario** (Gmail, Instagram, banco, etc.). Eso está dentro del JSON de la bóveda cifrada. Esto es una mejora respecto a algunos competidores históricos: en LastPass, las URLs de los servicios estaban en una zona menos protegida.

3. **Adivinar la contraseña maestra fácilmente.** El atacante puede intentar fuerza bruta offline contra la Caja A, pero:
 - El salt único impide rainbow tables.
 - Las 230.000 iteraciones de PBKDF2 hacen que cada intento cueste ~50-400 ms incluso en hardware especializado.
 - Una contraseña razonable (12+ chars con mezcla) tarda más que la edad del universo en romperse.
 - Una contraseña pésima (*123456*) cae en segundos. **Esto es responsabilidad del usuario.**
4. **Modificar los datos sin que el usuario lo note.** Si el atacante tuviera acceso de escritura y reemplazara la Caja A por otra distinta, en el siguiente login del usuario el descifrado fallaría con `AEADBadTagException`.
5. **Inferir nada combinando los datos de varios usuarios.** Las salts y DEKs son únicas e independientes; comprometer la cuenta de un usuario no aporta absolutamente nada para atacar la de otro.

¿Y si el atacante también obtiene el vault local del teléfono?

Eso requiere comprometer el teléfono además del servidor. En ese caso el atacante tendría dos copias del mismo blob cifrado (que sería redundante, no le da más información). Lo realmente peligroso sería que el atacante consiguiera la DEK en RAM del proceso de Clef desbloqueado — eso ya vimos en §10.5 y es un problema de cliente, no de servidor.

12. Limitaciones honestas del modelo

Una defensa técnica sería reconocer los puntos débiles. Estas son las limitaciones reales de Clef. En la defensa, mencionarlas tú primero te coloca por delante del tribunal.

12.1 Dependencia de la calidad de la contraseña maestra

PBKDF2 + salt es una defensa estructural, pero **no convierte una contraseña mala en buena**. Si el usuario elige `123456`, ningún esquema criptográfico le va a salvar. La app educa con un indicador de fortaleza, pero no fuerza una calidad mínima alta porque eso afectaría la usabilidad.

Mitigación posible: integrar un diccionario de contraseñas comunes (top 100k del HavelBeenPwned dataset, por ejemplo) y rechazar las que estén en él.

12.2 Iteraciones de PBKDF2

OWASP recomienda 600.000 iteraciones para PBKDF2-SHA256 en 2023. Clef usa 230.000 por compromiso de UX en móvil. Es defendible pero no óptimo.

Mitigación posible: migrar a Argon2id, que es más resistente a GPU/ASIC y permite ajustar coste de memoria, no solo de CPU. Argon2id es el estándar moderno y ha ganado el Password Hashing Competition (2015).

12.3 Cliente comprometido = todo comprometido

Zero-Knowledge protege el servidor, no el cliente. Si el dispositivo del usuario está infectado con malware avanzado (keylogger, screen recorder con permisos elevados), nada puede impedir que se capture la contraseña maestra cuando el usuario la introduce.

Mitigación posible: integración con el Trusted Execution Environment para procesamiento de la contraseña dentro del TEE. Tecnológicamente complejo y dependiente del hardware.

12.4 Metadatos visibles

Aunque el contenido está cifrado, el atacante con acceso a Firestore puede ver: - Cuándo se crea cada cuenta. - Cuándo se actualiza la bóveda (campo `version` y `updatedAt`). - Tamaño aproximado de la bóveda. - Frecuencia de uso (por las IPs registradas en `knownIps`).

Mitigación posible: padding aleatorio del blob de bóveda para ocultar el tamaño real, y desactivación opcional del registro de IPs.

12.5 Recuperación dependiente del PUK

Si el usuario pierde la contraseña Y el PUK, los datos se pierden definitivamente. No hay vía intermedia. Esto es una característica del modelo, no un fallo, pero hay que asumirlo y comunicarlo al usuario claramente.

Mitigación posible: un sistema de “social recovery” tipo Shamir’s Secret Sharing donde el usuario reparte porciones del PUK entre personas de confianza. Trabajo futuro.

12.6 PUK temporalmente en memoria

Aunque se hacen esfuerzos para zerizar el PUK tras mostrarlo (`Arrays.fill`), hay puntos donde Android puede haber hecho copias internas del String (como en `TextView`) que escapan al control de la app. Es una limitación de la JVM y de la API de Android, no del diseño de Clef.

12.7 No protege contra el sistema operativo

Si el sistema operativo del teléfono está comprometido (rootkit a nivel kernel), Clef no puede protegerse. Confía en la integridad del sistema base. *RootDetector* mitiga lo evidente pero no lo sofisticado.

13. Bibliografía y referencias

Estándares y especificaciones

- **NIST FIPS 197** (2001). *Advanced Encryption Standard (AES)*. <https://doi.org/10.6028/NIST.FIPS.197>
- **NIST SP 800-38D** (2007). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. <https://doi.org/10.6028/NIST.SP.800-38D>
- **NIST SP 800-132** (2010). *Recommendation for Password-Based Key Derivation Part 1: Storage Applications*. <https://doi.org/10.6028/NIST.SP.800-132>
- **NIST SP 800-57 Part 1** (Rev. 5, 2020). *Recommendation for Key Management Part 1: General*. <https://doi.org/10.6028/NIST.SP.800-57pt1r5>
- **NIST FIPS 180-4** (2015). *Secure Hash Standard (SHS)*. <https://doi.org/10.6028/NIST.FIPS.180-4>
- **RFC 2898** (2000). *PKCS #5: Password-Based Cryptography Specification Version 2.0*. Kaliski, B. <https://www.rfc-editor.org/rfc/rfc2898>
- **RFC 8018** (2017). *PKCS #5: Password-Based Cryptography Specification Version 2.1*. Moriarty, K. et al. <https://www.rfc-editor.org/rfc/rfc8018>

Guías de buenas prácticas

- **OWASP Password Storage Cheat Sheet** (2023). https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- **OWASP Cryptographic Storage Cheat Sheet** (2024). https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html
- **OWASP Mobile Application Security Verification Standard (MASVS) v2.0**. <https://mas.owasp.org/MASVS/>
- **OWASP Mobile Top 10** (2024). <https://owasp.org/www-project-mobile-top-10/>

Bibliografía académica

- Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice*, 7ª ed. Pearson. ISBN 978-0-13-444428-1. **Capítulos 5-7** sobre cifrado simétrico y modos de operación; **capítulo 11** sobre funciones hash.
- Ferguson, N., Schneier, B., Kohno, T. (2010). *Cryptography Engineering: Design Principles and Practical Applications*. Wiley. ISBN 978-0-470-47424-2.
- Boneh, D., Shoup, V. (2023). *A Graduate Course in Applied Cryptography*. v0.6. <https://toc.cryptobook.us/>
- Katz, J., Lindell, Y. (2020). *Introduction to Modern Cryptography*, 3ª ed. CRC Press.

Documentación técnica de plataforma

- Google. *Android Keystore System*. <https://developer.android.com/privacy-and-security/keystore>
- Google. *Cryptography on Android*. <https://developer.android.com/guide/topics/security/cryptography>
- Google. *Network Security Configuration*. <https://developer.android.com/privacy-and-security/security-config>

- Google. *Firestore Security Rules*.
<https://firebase.google.com/docs/firestore/security/get-started>

Análisis de gestores de contraseñas existentes

- **Bitwarden Security Whitepaper** (2024).
<https://bitwarden.com/help/bitwarden-security-white-paper/>
- **1Password Security Design** (2023).
<https://1passwordstatic.com/files/security/1password-white-paper.pdf>
- **Proton Pass Security Model** (2023). <https://proton.me/blog/proton-pass-security-model>
- Carnavalet, X., Mannan, M. (2014). *From Very Weak to Very Strong: Analyzing Password-Strength Meters*. NDSS Symposium.
- LastPass (2022). *Notice of Recent Security Incident*. Análisis posterior de la filtración. <https://blog.lastpass.com/posts/notice-of-recent-security-incident>

Recursos pedagógicos en español

- Khan Academy. *Criptografía*.
<https://es.khanacademy.org/computing/computer-science/cryptography>
- Aranda, J. (2020). *Criptografía: una introducción*. Universidad de Granada (apuntes abiertos).
- INCIBE-CERT. *Glosario de términos de ciberseguridad*.
<https://www.incibe.es/incibe-cert/glosario-de-terminos>

Sobre arquitectura Zero-Knowledge

- Goldwasser, S., Micali, S., Rackoff, C. (1989). *The Knowledge Complexity of Interactive Proof Systems*. SIAM Journal on Computing 18(1).
- Bellare, M., Rogaway, P. (2005). *Introduction to Modern Cryptography*.
Capítulo sobre key wrapping y envoltura de claves.

Sobre el caso LastPass

- Goodin, D. (2023). *LastPass users: Your info and password vault data are now in hackers' hands*. Ars Technica. <https://arstechnica.com/information->

[technology/2022/12/lastpass-says-hackers-have-obtained-vault-data-and-a-wealth-of-customer-info/](https://www.technology/2022/12/lastpass-says-hackers-have-obtained-vault-data-and-a-wealth-of-customer-info/)

Apéndice A: Glosario rápido

Término	Definición operativa
AEAD	Authenticated Encryption with Associated Data. Cifrado que garantiza confidencialidad + integridad. GCM es AEAD.
AES	Advanced Encryption Standard. Algoritmo simétrico estándar desde 2001.
DEK	Data Encryption Key. La clave que cifra los datos en sí (la bóveda).
Firestore	Base de datos en la nube de Firebase/Google. NoSQL, basada en documentos.
GCM	Galois/Counter Mode. Modo AEAD para cifrado por bloques.
HMAC	Hash-based Message Authentication Code. Construcción para autenticar mensajes con hash + clave.
IV	Initialization Vector. Valor aleatorio que se mezcla con cada operación de cifrado.
KDF	Key Derivation Function. Función para derivar claves de contraseñas. PBKDF2, scrypt, Argon2.
KEK	Key Encryption Key. Clave que cifra otras claves (no datos directamente).
PBKDF2	Password-Based Key Derivation Function 2. KDF estándar de NIST.

Término	Definición operativa
PUK	Personal Unblocking Key. Código de un solo uso para recuperación de cuenta en Clef.
Salt	Valor aleatorio público mezclado con la contraseña antes de derivar la clave.
TEE	Trusted Execution Environment. Entorno aislado en el SoC para procesamiento seguro.
Zero-Knowledge	Arquitectura donde el servidor no tiene capacidad alguna de leer los datos.
